

计算机组成原理
大作业
多周期与五级流水线 CPU 的复现与改进

鲁昕宁 2414015

2026 年 6 月 8 日

[https://github.com/SidneyLu/Computer-Organization-and-Design-NKU-
CS/tree/master/final_project](https://github.com/SidneyLu/Computer-Organization-and-Design-NKU-CS/tree/master/final_project)

1 实验目标

1. 找到多周期和五级流水线 CPU 原始代码和方案中存在的 bug 并修复，详细说明 bug 发现过程和修复过程及效果，保证现有的测试汇编程序执行正确。

BUG: CPU 实现中引入了同步 ROM 但仍按异步 ROM 的方式取指。

2. 仿照单周期 CPU 实验，自行用 MIPS 汇编编写一个完整具体的计算任务，把这个计算任务放到 coe 中交给流水线 CPU 执行。在流水线执行过程中会出现数据相关等情况，用前递与旁路解决数据冒险。

2 实验原理

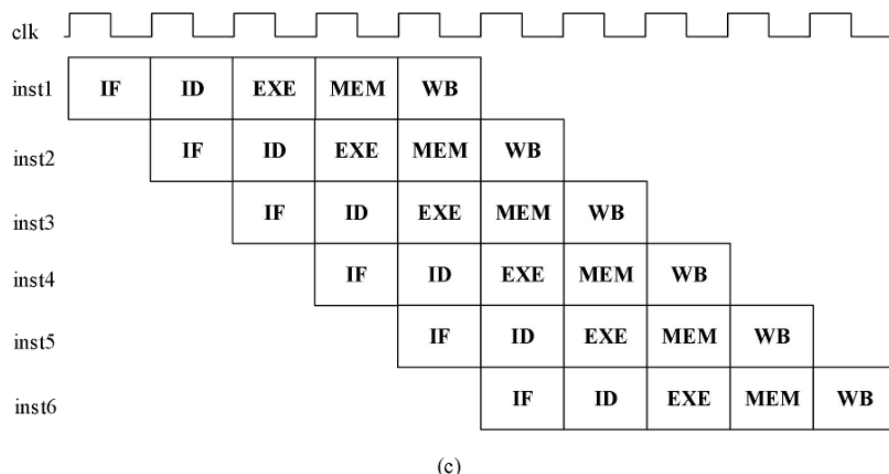


图 1: 5 级流水线 CPU 时空图

流水线（Pipeline）的引入是计算机体系结构发展史上的一次跃进。它的作用可以从以下几个维度来理解：

2.1 极大提升 CPU 的吞吐量（Throughput）

在没有流水线的设计中（如单周期或多周期 CPU），一条指令必须完整地经历“取指（IF）→ 译码（ID）→ 执行（EX）→ 访存（MEM）→ 写回（WB）”这五个阶段，下一条指令才能开始。这意味着在某一时刻，CPU 只有一小部分电路在工作，其余大部分硬件都在“闲置”。而引入流水线后，指令级并行程度得到提升：当第一条指令在执行（EX）时；第二条指令正在译码（ID）；第三条指令已经开始取指（IF）。当然流水线并没有缩短单条指令的执行时间（延迟 Latency），甚至由于增加了流水线寄存器，单条指令的延迟还会略微增加。但它让 CPU 在单位时间内能够完成的指令总数（吞吐量）呈倍数级增长。在理想状态下，每个时钟周期都能“吐出”一条执行完的指令。

2.2 拉高主频 (Clock Frequency)

评估 CPU 性能有一个公式：

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

CPI: 每条指令所需的平均时钟周期数。Clock Cycle Time: 时钟周期 (主频的倒数), 由电路中最长的那条路径 (关键路径) 的延时决定。在多周期 CPU 中, 为了让复杂的指令在单个时钟周期内不至于拖慢整体速度, 状态机和控制逻辑会变得极其繁琐; 而在单周期 CPU 中, 时钟周期必须满足最慢的那条指令 (通常是 lw), 导致主频根本无法提升。流水线把一条复杂的指令切分成 5 个 (或更多) 近似均等的微小阶段。时钟周期不再取决于整条指令的长度, 而仅仅取决于这 5 个阶段中最慢的那一个阶段的延时。这种 “化整为零” 的切分, 让关键路径大大缩短, 使得 CPU 的时钟频率 (主频) 可以轻松提升。

2.3 硬件资源利用率 (Hardware Utilization) 最大化

执行 addiu 的 ALU 运算时, 内存控制部件 (Memory) 完全在闲置。执行 lw 访存时, ALU 运算部件又在闲置。流水线设计让算术逻辑单元 (ALU)、指令寄存器 (IR)、数据存储器 (Data Memory) 和寄存器堆 (Register File) 在每一个时钟周期都处于全负荷运转状态。这种高强度的硬件复用, 用最经济的芯片面积换取了最大的算力输出。

2.4 催生了其他高性能处理器技术

流水线是所有现代高性能高级处理器技术的 “入场券”。没有流水线, 以下技术都无从谈起: 超流水线 (Super-pipelining): 将 5 级流水线继续细拆成 14 级甚至 31 级 (如 Intel Pentium 4), 把主频榨干到极致。超标量 (Superscalar): 既然一条流水线不够用, 那就并行摆放多条流水线, 实现每个周期同时发射、同时执行多条指令 (现代 PC 处理器多为 4-6 发射)。乱序执行 (Out-of-Order Execution) 与分支预测 (Branch Prediction): 专门为了解决流水线 “卡顿” (冒险/冲突) 而衍生出的顶级硬件优化技术。

当然, 世界上没有免费的午餐。流水线伴随着 “冒险 (Hazards)”:

- 结构冒险: 多个阶段同时争抢同一个硬件 (如同时读写内存)。
- 数据冒险: 后一条指令要用到前一条指令还没写回的脏数据 (数据冲突)
- 控制冒险: 遇到 beq、bgez 等跳转指令时, CPU 不知道该继续取哪里的指令

以上冒险导致流水线不得不面临 “刷洗 (Flush)” 或 “阻塞 (Stall)” 的惩罚现代 CPU 很大一部分设计精力, 其实都倾注在享受流水线高吞吐量的同时, 弥补这些由流水线带来的副作用, 如多发射、乱序执行、分支预测等。

3 实验过程

3.1 IP 核创建

按照实验指导手册 lab5 说明, 创建指令 ROM 和数据 RAM 的 IP 核。

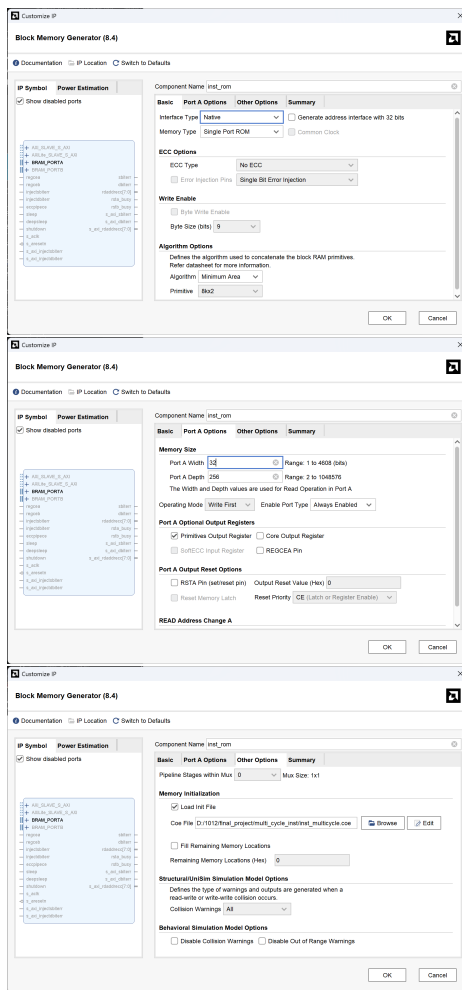


图 2: 指令 ROM IP 核的创建

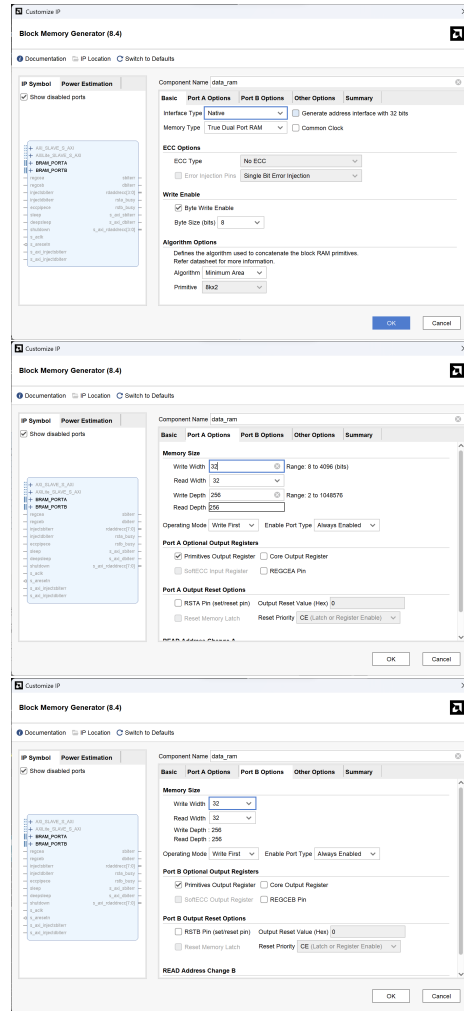


图 3: 数据 RAM IP 核的创建

3.2 多周期 CPU 分支指令与标准 MIPS 的差异

通过阅读多周期 CPU 测试程序的 MIPS 汇编代码，发现以下问题：按照标准 MIPS 实现，

$$\text{目标地址} = \text{当前指令地址 (PC)} + 4 + \text{立即数} \times 4 \quad (1)$$

但是多周期 CPU 的实现中是

$$\text{目标地址} = \text{当前指令地址 (PC)} + \text{立即数} \times 4 \quad (2)$$

```
// lab7/decode.v line 181
assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
```

这与流水线 CPU（采用标准实现）不同。

```
// lab8/decode.v line 198
wire [31:0] bd_pc;
assign bd_pc = pc + 3'b100;
// line 225
assign br_target[31:2] = bd_pc[31:2] + {{14{offset[15]}},
↪ offset};
// line 272 (延迟槽 PC+8)
assign alu_operand2 = inst_j_link ? 32'd8 :
                        inst_imm_zero ? {16'd0, imm} :
                        inst_imm_sign ? {{16{imm[15]}}, imm} :
                        ↪ rt_value;
```

按照标准 MIPS，多周期 CPU 的配套 coe 文件存在以下异常：

1.14H

bgez \$25,#16 跳转到 54H

此处跳转条件为真的情况下，跳转目标地址应为 $14+4=18(\text{Hex})=24(\text{Dec})$ ， $+(16 \times 4)=88(\text{Dec})=58(\text{Hex})$ 而非 $54(\text{Hex})$ 。直接跳转到 $58(\text{Hex})$ ，此时寄存器 \$12 未正确初始化从而程序会表现异常，要使程序行为正确，应修改立即数为 15。

2.30H

30H beq \$8, \$3,#2 跳转到 38H

同样是一个偏移量计算错误，正确行为：跳转到 38H，但立即数为 2 的情况下，由 MIPS 分支跳转指令的定义，可知目标地址变成了 $(30+4)+(2 \times 4)=3C(\text{Hex})$ ，应修改立即数为 1。

3.40H

40H bne \$10,\$5,#4 跳转到 50H

4.88H

同理，立即数应为 5。

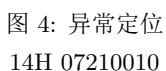
5.90H

要跳转到 AC(Hex)=172(Dec)，偏移量应为 $(172-(9 \times 16)-4)/4=6$ (Dec)。立即数应为 6。

为保证多周期 CPU 与标准 MIPS 定义一致，最终版本直接修改了 lab7 硬件，将分支目标地址计算改为

因此，`inst_multicycle.coe` 中对应的 5 处分支立即数也同步修正。这样处理后，多周期 CPU 和五级流水线 CPU 对分支偏移量的解释重新一致，后续调试与验证不再需要额外区分“非标准多周期版本”和“标准流水线版本”。

3.4.1 异常现象



6

PC 推进与预期不符，程序并没有稳定地进入文档描述的控制流，随后部分阶段信息开始错乱，导致整个测试程序无法继续正常跑通。

3.4.2 原因溯源：同步取指与原始取指逻辑不匹配

进一步检查 lab7 实现后发现，真正的问题出在取指级。当前工程中的 `inst_rom` 是同步读 IP 核，指令输出会晚一个时钟周期返回；但原始 `fetch` 模块仍按异步 ROM 的思路编写，直接把“当前 PC”和“当前看到的指令”打包送往译码级：

```
assign inst_addr = pc;
assign IF_ID_bus = {pc, inst};
```

这意味着送入 ID 级的 `pc` 与 `inst` 并不属于同一次取指请求，而是发生了错位：

- `pc` 是本拍正在请求的地址；
- `inst` 却是上一拍地址对应的返回数据。

一旦这种错位传播到分支指令，就会出现“译码看到的指令”和“用于计算跳转目标的 PC”不一致的情况。14H 附近之所以最早暴露异常，正是因为这里第一次明显依赖正确的分支目标地址与控制流切换；一旦取值错位，后续状态机推进和 PC 更新就会偏离文档描述。

3.4.3 修复方案

针对这一问题，最终在 lab7 中采取了两点修复，且都不改变原测试文档语义：

- 在 `fetch` 级加入 `pc_buf`，用它保存与同步 `inst_rom` 返回数据相对应的请求地址，再将 `IF_ID_bus` 改为 `{pc_buf, inst}`；
- 调整 `IF_over` 的握手机制，在发起新一次取指请求后额外等待同步 ROM 返回，避免状态机过早把错位数据送入 ID 级。

修复后的关键代码如下：

```
// fetch.v
reg [31:0] pc_buf;
assign inst_addr = pc;
assign IF_ID_bus = {pc_buf, inst};

always @(posedge clk) begin
    if (!resetn) begin
        pc <= 32'd0;
        pc_buf <= 32'd0;
    end else if (next_fetch) begin
```

```

        pc <= next_pc;
    end
    if (IF_valid) begin
        pc_buf <= pc;
    end
end
end

```

3.4.4 修复结果

修复后，多周期 CPU 重新满足原测试文档的要求：14H bgez \$25,#16 能够按文档跳转到 54H，而不是因为取指错位而故障。由此可以说明，原始多周期 CPU 的主要故障是同步指令 ROM 已经引入，但取指级仍沿用异步 ROM 写法，两者时序不匹配导致了 pc 与 inst 配对错误。修正同步取指后，该异常得到消除。

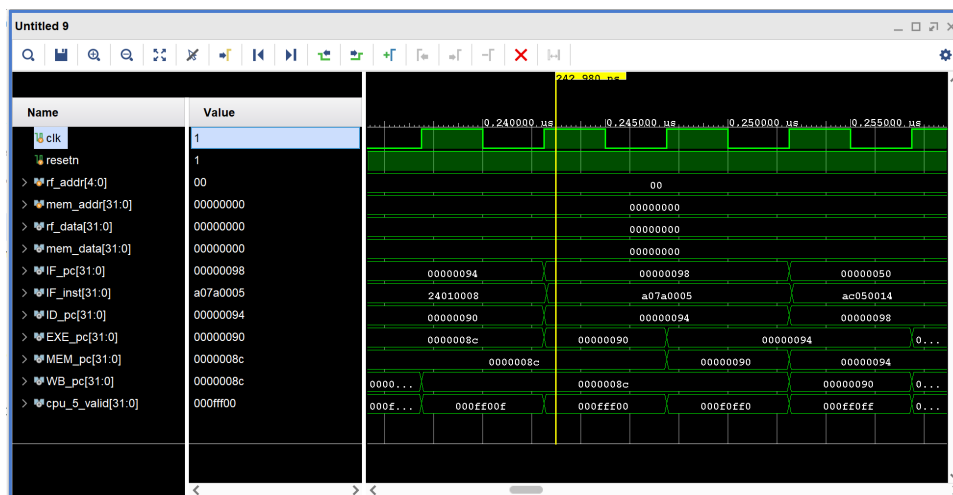


图 5: 修复后的多周期 CPU，可以正常运行原始测试程序。

3.5 原始流水线 CPU 的异常

3.5.1 异常现象

在对原始五级流水线 CPU 进行仿真时，程序在 90H 附近出现异常。按照测试文档，

```

8CH  addiu  $27,$26,#68    -> [$27] = 0000_0050H
90H  jalr   $27             -> 跳转到 50H, [$31] = 0000_0098H
94H  addiu  $1,$0,#8       -> 作为延迟槽执行

```

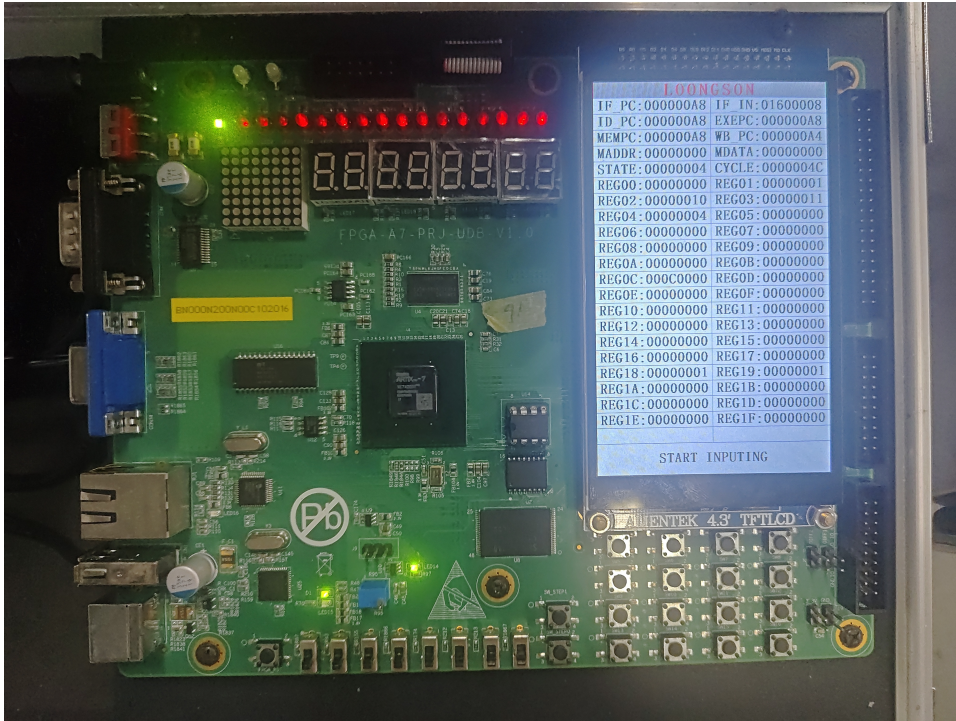


图 6: 复现的多周期 CPU（上箱验证）

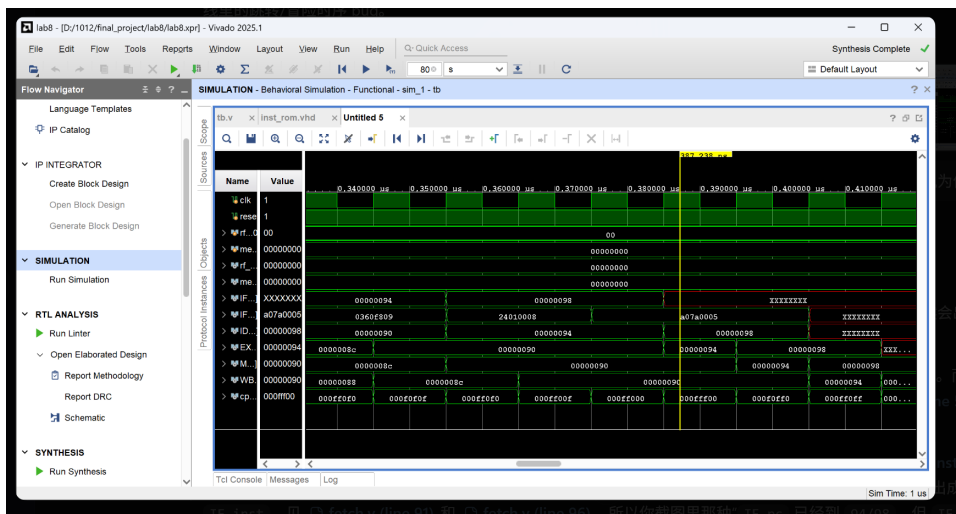


图 7: 异常定位
90H 0360F809
94H 24010008

因此正确的动态执行顺序应为

84H → 88H → 8CH → 90H → 94H(延迟槽) → 50H → ⋯ → 7CH → 80H(延迟槽) → 98H
(4)

然而原始实现的波形显示: 90H jalr \$27 之后并没有按要求转入 50H, 而是让

98H/9CH 等顺序路径上的指令提前进入流水线，随后 IF/ID/EXE/MEM/WB 级的 PC 与部分数据逐步扩散为 X，程序最终无法继续正常执行。

3.5.2 初步误判：怀疑 RAM 端口模式或 IP 配置异常

最初曾怀疑异常与 data_ram 的双端口配置有关，尤其是端口 B 的 READ_FIRST/WRITE_FIRST 模式可能会影响访存结果，进一步传导到控制流。但结合顶层连线可以排除这一点：CPU 在执行过程中实际使用的是 Port A，Port B 仅用于板上显示和调试观测，因此即使 Port B 的模式设置不同，也不会直接导致 jalr 跳转目标错误。也就是说，这次故障并不是单纯的数据 RAM 参数问题。

3.5.3 对照测试程序后的重新定位

对照《5 级流水线 CPU 测试程序详解》，可以发现 98H sb \$26, #5(\$3) 并不应该在 90H jalr 之后立即出现。它本应在后续执行 50H $\rightarrow$... $\rightarrow$ 7CH jal #38 之后，作为新的跳转目标重新进入流水线。因此，90H 之后若直接顺序执行到 98H/9CH，就说明控制流在 jalr 处已经故障。

进一步沿着测试程序的数据依赖关系向后检查，可得到一条清晰的异常传播链：

90H 50H $\Rightarrow$ 6CH lw \$10, #28(\$0) $\Rightarrow$ \$10 未初始化 (5)
 $\Rightarrow$ 9CH sltu \$13, \$10, \$3 得到 X $\Rightarrow$ A0H bgtz \$13, #2 分支判断被污染 $\Rightarrow$ PC 扩散为 X (6)

其中，9CH sltu \$13, \$10, \$3 使用的 \$10 本应由前面 6CH lw \$10, #28(\$0) 写入；一旦程序没有先跳回 50H 并沿正确路径执行到 6CH，那么 \$10 就处于未定义状态，随后 \$13、分支条件以及 PC 都会被逐级污染为 X。

3.5.4 原因溯源

综合 RTL 与测试程序可知，该异常不是单一模块，而是下列三个因素共同造成的：

- inst_rom 为同步读，指令返回晚一拍；但原 fetch 级直接把当前 pc 与返回的 inst 打包，导致“指令”和“对应地址”在时序上错位。
- jalr/jr 以及分支类指令在 ID 级就要决定控制流，但原实现对 ID 级源操作数缺少完整的前递/旁路路径，\$27 尚未稳定时就可能过早形成 j_target。
- 一旦 90H 没有正确跳到 50H，顺序路径上的 98H/9CH/A0H 会提前执行，从而触发未初始化寄存器参与比较和分支判断，最终把 PC 污染为 X。

因此，原始流水线 CPU 在这里暴露出的问题是：

控制相关与同步取指时序发生了错误

3.6 五级流水线 CPU 的改进：旁路

本次对五级流水线 CPU 的修复分成了两个阶段：**第一阶段**修复 ID/EXE 之间的数据相关；**第二阶段**修复同步 `inst_rom` 条件下 IF 级地址与指令返回结果不对齐的问题。整个过程中，没有改变原有的延迟槽机制，也没有改变 `jal/jalr` 写回 `PC+8` 的语义，而是在保留这些既有约束的前提下，使控制流和数据流重新与测试程序文档一致。

3.6.1 第一阶段：EXE 级前递 + ID 级控制相关旁路

最先完成的改进，是把原本“只要相关就统一停顿”的策略调整为“EXE 级优先前递，只有确实无法前递时再停顿”。前递相关的元信息主要在 `pipeline_cpu.v` 中整理。设计中从 `ID_EXE_bus_r` 与 `EXE_MEM_bus_r` 中拆出目的寄存器号、寄存器写使能以及结果类型，并据此构造两类控制信号。第一类是 `EXE_slow_result` 和 `MEM_slow_result`，用于标记当前阶段结果虽然最终会写回寄存器，但此时还不能安全地被后继指令直接使用；在本实现中，`load`、`mfhi`、`mflo`、`mfc0` 都被归入这一类慢结果。第二类是可直接用于前递的旁路信息，即 `MEM_fwd_wen`、`MEM_fwd_wdest` 和 `MEM_fwd_data`；WB 级则直接复用 `rf_wen`、`rf_wdest` 和 `rf_wdata` 作为第二路前递源。

真正的数据选择发生在 `exe.v`。EXE 级会分别比较当前指令的 `rs` 和 `rt` 与 `MEM_fwd_wdest`、`WB_fwd_wdest` 是否匹配；若命中 MEM 前递源，则优先选用 `MEM_fwd_data`；若 MEM 未命中但 WB 命中，则退而选用 `WB_fwd_data`；只有两级都未命中时，才回退到寄存器堆读出的原始值。这套选择逻辑同时服务于 `alu_operand1`、`alu_operand2` 和 `store_data`，因此不仅支持普通的 ALU $\rightarrow$ ALU 前递，也支持 ALU $\rightarrow$ store 前递。

在进一步排查 90H `jalr $27` 异常后，又对 ID 级控制相关补上了必要的 MEM/WB 前递路径。也就是说，`jr`、`jalr`、`beq`、`bne`、`bgez`、`bgtz`、`blez` 和 `bltz` 不再只依赖寄存器堆的原始读出值，而是可以优先使用已经在 MEM 或 WB 阶段形成的稳定结果。与此同时，`regfile` 中还加入了同拍写回-读出旁路，以避免 WB $\rightarrow$ ID 边界读到旧值。最终版本对 ID 级控制相关加入必要的 MEM/WB 前递；对 EXE 尚未完成或慢结果相关的情况仍采用停顿策略。

3.6.2 第二阶段：修复同步 ROM 下的取指对齐问题

仅有第一阶段仍不足以彻底解决问题。前文的分析发现，原实现的 `inst_rom` 为同步读，即 PC 送入 ROM 后，下一拍才返回对应指令；但原 `fetch` 级仍直接把“当前请求地址 `pc`”与“本拍返回指令 `inst`”打包为 `IF_ID_bus`，从而造成地址与指令错位。这正是 `jalr` 之后控制流判断与后续波形显示持续混乱的重要原因。

为此，`fetch` 级被改为“返回指令配对其请求地址”的结构：在发起取指请求时，先将该请求地址保存到缓冲寄存器；待同步 ROM 在下一拍返回 `inst`

时，再把该缓冲地址与 `inst` 组合后送往 ID 级。换言之，进入 `IF_ID_bus` 以及用于展示的 `IF_pc`，都不再直接使用当前请求 PC，而是使用与本拍返回指令真正对应的 PC。这样处理后，ID 级的分支/跳转判断、波形观测与测试程序中的“指令地址-语义”关系重新对齐。

3.6.3 修复后的行为与效果

修复完成后，流水线 CPU 在 90H `jalr $27` 处应满足以下关键行为：

- 90H `jalr` 之后仅执行 94H `addiu $1,$0,#8` 作为延迟槽；
- 随后真正转入 50H，而不是顺序落到 98H；
- 8CH `addiu $27,$26,#68` 执行后应得到 `$27 = 0x00000050`，且 `jalr` 按原语义写回 `$31 = 0x00000098`；
- 后续 6CH `lw`、9CH `sltu`、A0H `bgtz` 的动态依赖恢复正常，不再因未初始化寄存器参与比较而污染控制流；
- 波形中的 `IF_pc`、`ID_pc`、`EXE_pc`、`MEM_pc`、`WB_pc` 不应在该处扩散为 X。
- 已支持的主要场景包括：`ALU → ALU`、跨 1 条指令的 `ALU → ALU`、`ALU → store`、ID 级 `jr/jalr/branch` 对 MEM/WB 稳定结果的前递使用。
- 仍需停顿的场景包括：`load-use`、EXE 级尚未完成的控制相关、`mfhi/mflo/mfc0` 等慢结果相关。
- 乘法仍按原多周期乘法部件的完成时刻决定 `EXE_over`，它不属于“单拍即可直接旁路”的普通 ALU 结果。

上述改进的最终效果是：普通整数运算之间不再需要大量人为插入 `nop`，`store` 指令可以直接使用前面刚算出的结果；与此同时，原本在 90H `jalr` 附近暴露出的控制相关与同步取指错位问题也得到修复。这样既保留了原有流水线的延迟槽语义和异常处理路径，又保证了测试文档中的控制流顺序与仿真结果重新一致。

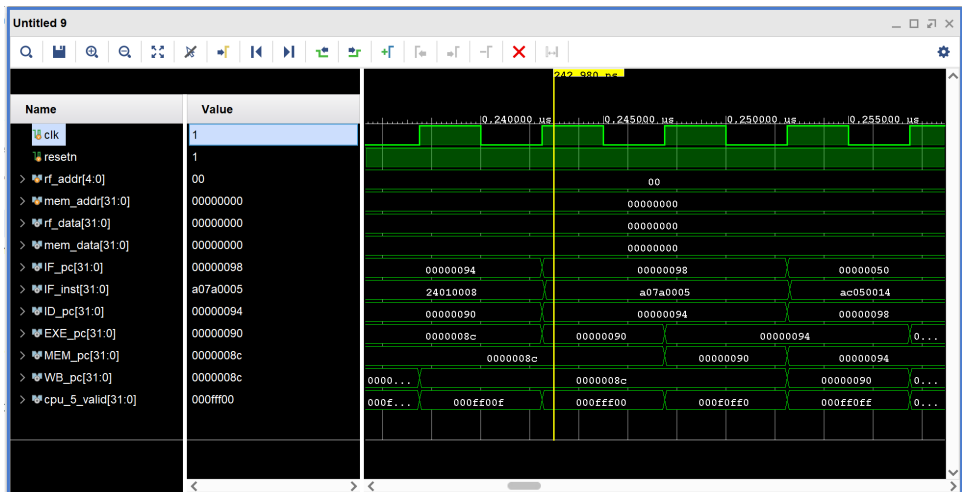


图 8: 修复后的流水线 CPU，可以正常运行原始测试程序。
注意此时 90H 指令可以正常跳转到 50H AC050014

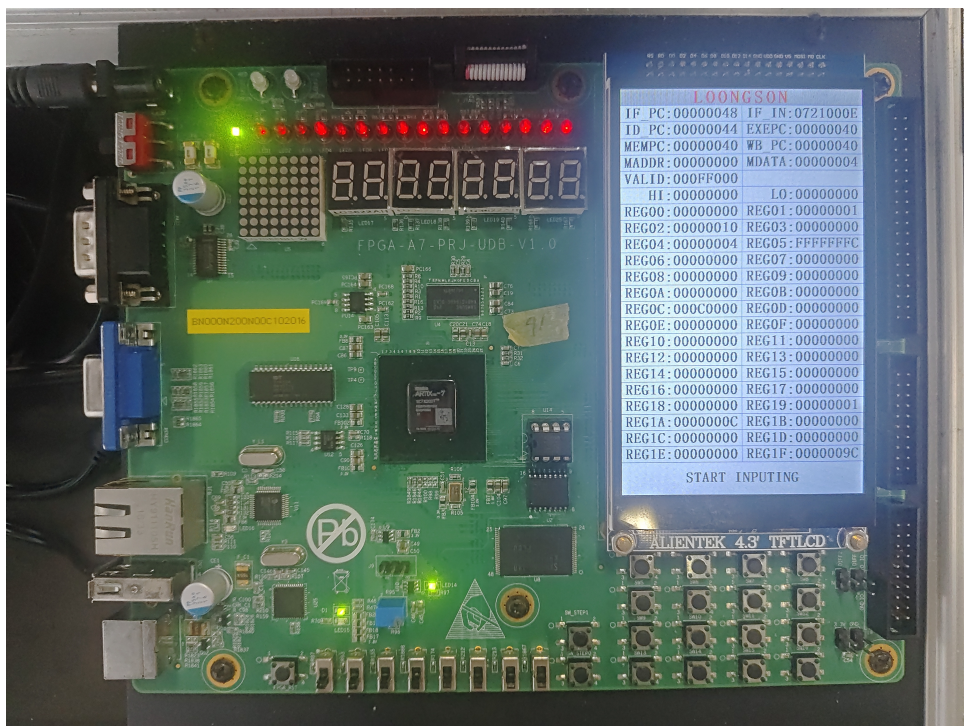


图 9: 在复现基础上加入前递与旁路的流水线 CPU（上箱验证）

4 关键代码

4.1 取指 IF

```
module fetch(  
    input          clk,  
    input          resetn,  
    input          IF_valid,  
    input          next_fetch, // 流水线暂停/继续控制  
    input [32:0] jbr_bus,      // 分支跳转总线: {taken,  
    ↪ target}  
    input [32:0] exc_bus,      // 异常处理总线: {valid, pc}  
    output reg      IF_over,   // 阶段完成握手  
    output [63:0] IF_ID_bus    // IF→ID 流水线寄存器  
);  
// 核心: PC 更新逻辑 (优先级: 异常 > 分支跳转 > 顺序执行)  
reg [31:0] pc;  
reg [31:0] pc_buf; // 同步 ROM 延迟补偿  
wire [31:0] seq_pc;  
wire [31:0] next_pc;  
wire        jbr_taken;  
wire [31:0] jbr_target;  
assign {jbr_taken, jbr_target} = jbr_bus;  
wire        exc_valid;  
wire [31:0] exc_pc;  
assign {exc_valid, exc_pc} = exc_bus;  
  
assign seq_pc = pc + 32'd4; // 顺序 PC+4  
assign next_pc = exc_valid ? exc_pc :  
                  jbr_taken ? jbr_target : seq_pc;  
always @(posedge clk) begin  
    if (!resetn) begin  
        pc <= `STARTADDR;  
        pc_buf <= `STARTADDR;  
    end  
    else if (next_fetch) begin // 只有允许取指时才更新 PC  
        pc_buf <= pc;  
        pc <= next_pc;  
    end  
end  
end
```

```

// 流水线握手：同步 ROM 需要 1 周期延迟
always @(posedge clk) begin
    if (!resetn || next_fetch)
        IF_over <= 1'b0;
    else
        IF_over <= IF_valid;
end
// IF→ID 总线：{指令对应的 PC, 指令}
assign IF_ID_bus = {pc_buf, inst};
endmodule

```

4.2 译码 ID

```

module decode(
    input                ID_valid,
    input                [ 63:0] IF_ID_bus_r,
    input                [ 31:0] rs_value,
    input                [ 31:0] rt_value,
    output               [ 4:0] rs,
    output               [ 4:0] rt,
    output               [ 32:0] jbr_bus,
    output               ID_over,
    output               [212:0] ID_EXE_bus,
    // 数据冒险检测输入（来自后续阶段）
    input               IF_over,
    input               [ 4:0] EXE_wdest,
    input               EXE_slow_result, // 慢指令标记
    ⇨ (load/mfhi/mflo)
    input               [ 4:0] MEM_wdest,
    input               MEM_slow_result,
    input               MEM_fwd_wen,
    input               [ 4:0] MEM_fwd_wdest,
    input               [ 31:0] MEM_fwd_data,
    input               WB_fwd_wen,
    input               [ 4:0] WB_fwd_wdest,
    input               [ 31:0] WB_fwd_data,
    input               [ 4:0] WB_wdest
);
wire [31:0] pc;
wire [31:0] inst;

```

```

assign {pc, inst} = IF_ID_bus_r;
// 指令字段解析（仅保留冒险检测需要的部分）
wire [4:0] rs;
wire [4:0] rt;
wire [4:0] rd;
assign rs = inst[25:21];
assign rt = inst[20:16];
assign rd = inst[15:11];
// ===== 核心1: ID阶段分支前递
// =====
// 分支/JR 指令在 ID 阶段就需要寄存器值，必须单独前递
wire rs_from_mem_id;
wire rs_from_wb_id;
wire rt_from_mem_id;
wire rt_from_wb_id;
wire [31:0] rs_value_id;
wire [31:0] rt_value_id;
// 前递条件：MEM/WB 阶段有写使能，且目标寄存器匹配（排除 $0）
assign rs_from_mem_id = MEM_fwd_wen & (rs != 5'd0) & (rs ==
    ↪ MEM_fwd_wdest);
assign rs_from_wb_id = WB_fwd_wen & (rs != 5'd0) & (rs ==
    ↪ WB_fwd_wdest);
assign rt_from_mem_id = MEM_fwd_wen & (rt != 5'd0) & (rt ==
    ↪ MEM_fwd_wdest);
assign rt_from_wb_id = WB_fwd_wen & (rt != 5'd0) & (rt ==
    ↪ WB_fwd_wdest);
// 前递优先级：MEM > WB > 寄存器堆原始值
assign rs_value_id = rs_from_mem_id ? MEM_fwd_data :
    rs_from_wb_id ? WB_fwd_data : rs_value;
assign rt_value_id = rt_from_mem_id ? MEM_fwd_data :
    rt_from_wb_id ? WB_fwd_data : rt_value;
// 分支跳转逻辑（基于前递后的值）
wire j_taken;
wire br_taken;
wire jbr_taken;
wire [31:0] jbr_target;
assign jbr_taken = (j_taken | br_taken) & ID_over;
assign jbr_bus = {jbr_taken, jbr_target};

```

```

// ===== 核心2: 数据冒险检测与流水线暂停
↪ =====
// 标记哪些指令在哪个阶段需要读取寄存器
wire rs_read_id;    // ID 阶段读 rs (分支/JR)
wire rt_read_id;    // ID 阶段读 rt (BEQ/BNE)
wire rs_read_exe;   // EXE 阶段读 rs (大部分指令)
wire rt_read_exe;   // EXE 阶段读 rt (R 型/SW)
// 检测寄存器依赖
wire exe_dep_rs = (rs != 5'd0) & (rs == EXE_wdest);
wire exe_dep_rt = (rt != 5'd0) & (rt == EXE_wdest);
wire mem_dep_rs = (rs != 5'd0) & (rs == MEM_wdest);
wire mem_dep_rt = (rt != 5'd0) & (rt == MEM_wdest);
wire wb_dep_rs  = (rs != 5'd0) & (rs == WB_wdest);
wire wb_dep_rt  = (rt != 5'd0) & (rt == WB_wdest);
// 暂停条件: 无法前递的依赖 (慢指令)
wire rs_wait_id = rs_read_id & (exe_dep_rs | (mem_dep_rs &
↪ MEM_slow_result));
wire rt_wait_id = rt_read_id & (exe_dep_rt | (mem_dep_rt &
↪ MEM_slow_result));
wire rs_wait_exe = rs_read_exe & ((exe_dep_rs & EXE_slow_result)
↪ | (mem_dep_rs & MEM_slow_result));
wire rt_wait_exe = rt_read_exe & ((exe_dep_rt & EXE_slow_result)
↪ | (mem_dep_rt & MEM_slow_result));
wire rs_wait = rs_wait_id | rs_wait_exe;
wire rt_wait = rt_wait_id | rt_wait_exe;
// 流水线握手: 无等待时阶段完成
assign ID_over = ID_valid & ~rs_wait & ~rt_wait & (~inst_jbr |
↪ IF_over);
// ID→EXE 总线 (包含所有控制信号和数据)
assign ID_EXE_bus = {
    // 控制信号
    multiply, mthi, mtlo, alu_control,
    // 数据
    rs_value, rt_value, imm_ext, sa_ext,
    // 前递需要的寄存器编号
    rs, rt,
    // 后续控制
    mem_control, mfhi, mflo, mtc0, mfc0,
    cp0r_addr, syscall, eret, rf_wen, rf_wdest,

```

```

        pc
    };
endmodule

```

4.3 执行 EX

```

module exe(
    input          EXE_valid,
    input    [212:0] ID_EXE_bus_r,
    output          EXE_over,
    output    [153:0] EXE_MEM_bus,

    // 前递输入 (来自 MEM/WB 阶段)
    input          MEM_fwd_wen,
    input    [ 4:0] MEM_fwd_wdest,
    input    [31:0] MEM_fwd_data,
    input          WB_fwd_wen,
    input    [ 4:0] WB_fwd_wdest,
    input    [31:0] WB_fwd_data,
    output    [ 4:0] EXE_wdest // 输出写目标, 供ID阶段冒险检测
);
// 总线解析 (仅保留前递需要的部分)
wire [31:0] rs_raw_value;
wire [31:0] rt_raw_value;
wire [4:0]  rs;
wire [4:0]  rt;
wire        rf_wen;
wire [4:0]  rf_wdest;
assign {
    multiply, mthi, mtlo, alu_control,
    rs_raw_value, rt_raw_value, imm_ext, sa_ext,
    op1_sel, op2_sel,
    rs, rt, // 前递需要的寄存器编号
    mem_control, mfhi, mflo, mtc0, mfc0,
    cp0r_addr, syscall, eret, rf_wen, rf_wdest,
    pc
} = ID_EXE_bus_r;
// ===== 核心: EXE阶段通用前递机制
// =====
wire rs_from_mem;

```

```

wire rs_from_wb;
wire rt_from_mem;
wire rt_from_wb;
wire [31:0] rs_value;
wire [31:0] rt_value;
// 前递条件
assign rs_from_mem = MEM_fwd_wen & (rs != 5'd0) & (rs ==
    ↪ MEM_fwd_wdest);
assign rs_from_wb = WB_fwd_wen & (rs != 5'd0) & (rs ==
    ↪ WB_fwd_wdest);
assign rt_from_mem = MEM_fwd_wen & (rt != 5'd0) & (rt ==
    ↪ MEM_fwd_wdest);
assign rt_from_wb = WB_fwd_wen & (rt != 5'd0) & (rt ==
    ↪ WB_fwd_wdest);
// 前递优先级: MEM > WB > 原始值
assign rs_value = rs_from_mem ? MEM_fwd_data :
    rs_from_wb ? WB_fwd_data : rs_raw_value;
assign rt_value = rt_from_mem ? MEM_fwd_data :
    rt_from_wb ? WB_fwd_data : rt_raw_value;
// ALU 操作数选择 (使用前递后的值)
wire [31:0] alu_operand1 = (op1_sel == 2'd2) ? pc :
    (op1_sel == 2'd1) ? sa_ext :
    ↪ rs_value;
wire [31:0] alu_operand2 = (op2_sel == 2'd2) ? 32'd8 :
    (op2_sel == 2'd1) ? imm_ext :
    ↪ rt_value;
// 流水线握手: 乘法指令需要等待, 其他立即完成
assign EXE_over = EXE_valid & (~multiply | mult_end);
// 输出写目标寄存器, 供 ID 阶段检测冒险
assign EXE_wdest = rf_wdest & {5{EXE_valid}};
// EXE→MEM 总线
assign EXE_MEM_bus = {
    mem_control, store_data, exe_result, lo_result,
    hi_write, lo_write, mfhi, mflo, mtc0, mfc0,
    cp0r_addr, syscall, eret, rf_wen, rf_wdest,
    pc
};

endmodule

```

4.4 访存 MEM

```
module mem(
    input          clk,
    input          MEM_valid,
    input  [153:0] EXE_MEM_bus_r,
    output         MEM_over,
    output  [117:0] MEM_WB_bus,

    // 前递输出 (供 ID/EXE 阶段使用)
    output         MEM_fwd_wen,
    output  [ 4:0] MEM_fwd_wdest,
    output  [31:0] MEM_fwd_data,
    output  [ 4:0] MEM_wdest, // 供 ID 阶段冒险检测
    output         MEM_slow_result // 标记慢指令
);
// 总线解析
wire [3:0] mem_control;
wire [31:0] exe_result;
wire      rf_wen;
wire [4:0] rf_wdest;
assign {
    mem_control, store_data, exe_result, lo_result,
    hi_write, lo_write, mfhi, mflo, mtc0, mfc0,
    cp0r_addr, syscall, eret, rf_wen, rf_wdest,
    pc
} = EXE_MEM_bus_r;
wire inst_load = mem_control[3];
// ===== 核心: MEM阶段前递输出
// =====
// 前递使能: MEM 阶段有效且需要写回寄存器
assign MEM_fwd_wen = rf_wen & MEM_valid;
assign MEM_fwd_wdest = rf_wdest;
// 前递数据: load 指令需要等待 RAM 返回, 其他直接用 EXE 结果
assign MEM_fwd_data = inst_load ? load_result : exe_result;
// 慢指令标记: load 指令需要 1 周期延迟, 无法立即前递到 ID 阶段
assign MEM_slow_result = inst_load & MEM_valid;
// 流水线握手: load 指令需要等待 RAM 返回
reg MEM_valid_r;
always @(posedge clk) begin
```



```

        if (MEM_allow_in)
            MEM_valid_r <= 1'b0;
        else
            MEM_valid_r <= MEM_valid;
    end
    assign MEM_over = inst_load ? MEM_valid_r : MEM_valid;
    // 输出写目标寄存器, 供 ID 阶段检测冒险
    assign MEM_wdest = rf_wdest & {5{MEM_valid}};
    // MEM→WB 总线
    assign MEM_WB_bus = {
        rf_wen, rf_wdest, mem_result, lo_result,
        hi_write, lo_write, mfhi, mflo, mtc0, mfc0,
        cp0r_addr, syscall, eret, pc
    };
endmodule

```

4.5 写回 WB

```

module wb(
    input          WB_valid,
    input  [117:0] MEM_WB_bus_r,
    output         rf_wen,
    output  [ 4:0] rf_wdest,
    output  [31:0] rf_wdata,
    output         WB_over,

    // 前递输出 (供 ID/EXE 阶段使用)
    output         WB_fwd_wen,
    output  [ 4:0] WB_fwd_wdest,
    output  [31:0] WB_fwd_data,
    output  [ 4:0] WB_wdest,  // 供 ID 阶段冒险检测
    output         cancel     // 流水线取消信号 (异常/ERET)
);
// 总线解析
wire      wen;
wire [4:0] wdest;
wire [31:0] mem_result;
wire      syscall;
wire      eret;
assign {

```

```

        wen, wdest, mem_result, lo_result,
        hi_write, lo_write, mfhi, mflo, mtc0, mfc0,
        cp0r_addr, syscall, eret, pc
    } = MEM_WB_bus_r;
// ===== 核心: WB阶段前递输出
↳ =====
assign WB_fwd_wen = wen & WB_valid;
assign WB_fwd_wdest = wdest;
assign WB_fwd_data = rf_wdata;
// 流水线握手: WB 阶段总是立即完成
assign WB_over = WB_valid;
// 寄存器堆写回
assign rf_wen    = wen & WB_over;
assign rf_wdest = wdest;
assign rf_wdata = mfhi ? hi :
                  mflo ? lo :
                  mfc0 ? cp0r_rdata : mem_result;
// 输出写目标寄存器, 供 ID 阶段检测冒险
assign WB_wdest = rf_wdest & {5{WB_valid}};
// 流水线取消: SYSCALL 或 ERET 指令清空整个流水线
assign cancel = (syscall | eret) & WB_over;
endmodule

```

5 计算任务: 斐波那契数列特定项的计算

计算任务最终采用和单周期 CPU 实验中固定输入的迭代版 Fib(10) 程序, 源码单独保存在 final_project/pipeline_inst/fib10_pipeline.asm 中。考虑到流水线 CPU 复位入口地址固定为 34H, 新的 inst_pipeline.coe 前部保留了 00H-30H 的填充区, 使程序从 34H 开始执行。在多周期和流水线版本的实现中, 均统计完成斐波那契数列第 10 项所耗费的时钟周期数并加以显示。程序的核心循环如下:

```

addiu $11, $0, 9      # remaining iterations
loop:
    addiu $11, $11, -1
    addu  $12, $9,  $10
    addu  $13, $12, $0
    addu  $9,  $10, $0
    addu  $10, $12, $0
    bgtz  $11, loop

```

```
addu $14, $10, $0    # delay slot
```

该版本主要覆盖了以下相关场景：

- addu \$12,\$9,\$10 -> addu \$13,\$12,\$0: ALU→ALU 前递；
- addu \$12,\$9,\$10 -> addu \$10,\$12,\$0: 跨 1 条指令的 ALU 前递；
- addu \$14,\$10,\$0 -> sw \$14,4(\$0): ALU→store 前递；
- bgtz 保留原有延迟槽，末尾进入稳定自循环，便于仿真和上板观察。

5.1 验证结果

5.1.1 多周期 CPU 上的斐波那契计算程序

```
Mem[1] = 0x00000037
```

```
$10 = 0x00000037
```

```
fib_done = 1
```

```
cycle_count = 273
```

5.1.2 流水线 CPU 上板验证

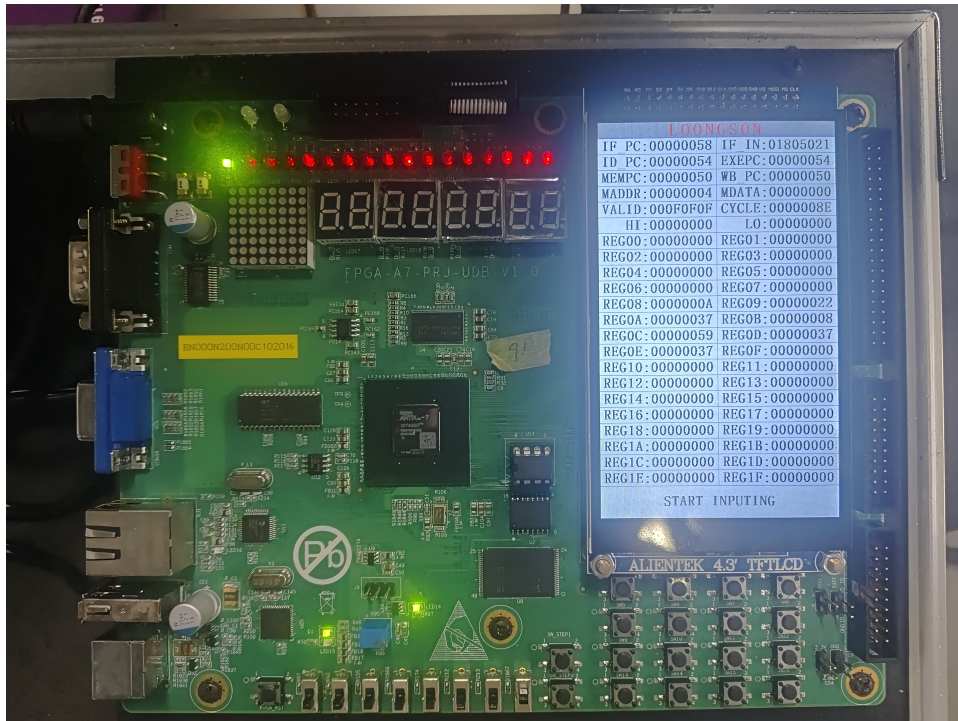


图 10: 五级流水线 CPU 计算 Fib(10)

可以从 LCD 屏上读取，计算得到斐波那契数列第 10 项，也即 55（十六进制下为 37）并将其写回内存 Mem[1]，五级流水线 CPU 消耗的周期数为 8E（142）。这段 Fib(10) 在结果写回前，动态执行的指令大约是：初始化 4 条，循

环体 7 条，执行 9 次，共 63 条，最后 sw1 条合计约 68 条。指令 ROM 是同步读，PC 更新后，下一拍才能拿到指令。所以 IF 级本身就要 2 个周期，这意味着当前流水线的取指吞吐率上限不是 1 IPC，而更接近每 2 个周期取到 1 条新指令，于是理论下界大概就是： $68 \times 2 + 4 = 140$ 。

实验结果证明了以流水线的方式提高指令级并行度可以缩短程序的运行时间，提升硬件的利用效率。

6 实验收获

- 实操了同步 ROM 和 RAM IP 核的创建
- 尝试了将汇编程序转换成 coe 文件预写入指令 ROM
- 深入学习了五级流水线 CPU 的硬件描述实现
- 系统运用了之前所学的 verilog 知识